



Engenharia de Software  
Dynamic Programming  
Aula 2 – 2º semestre - Recursão

Prof. Dr. Francisco Elânio

# Objetivos da Aula

- **Compreender o conceito de recursão**
- **Identificar caso base e caso recursivo**
- **Implementar funções recursivas em Python**
- **Aplicar exemplos clássicos: fatorial e Fibonacci**

# O que é Recursão?

É uma técnica onde uma função chama a si mesma para resolver um problema, dividindo-o em subproblemas menores e mais simples. É como um ciclo onde a função se repete até atingir uma condição de parada.

**Uso:** problemas naturalmente divisíveis em subproblemas menores.

# Fatorial

O fatorial de um número  $n$  é o produto de todos os inteiros positivos menores ou iguais a  $n$ .

A função recursiva para calcular o fatorial seria:

```
fatorial(n):  
    se n == 0:  
        retorna 1 # Caso base  
    senão:  
        retorna n * fatorial(n-1) # Chamada recursiva
```

*O caso base é quando  $n$  é igual a 0, onde a função retorna 1. Caso contrário, a função calcula  $n$  multiplicado pelo fatorial de  $n-1$ , chamando a si mesma com o valor reduzido.*



# Definição Matemática: Fatorial

O fatorial de um número é igual à multiplicação desse número pelos seus antecessores maiores que zero.

$$n! = n \times (n-1)!$$

$$0! = 1 \text{ (caso base)}$$

$$\begin{aligned} 0! &= 1 \\ 1! &= 1 \\ 2! &= 2 \times 1 \\ 3! &= 3 \times 2 \times 1 \\ 4! &= 4 \times 3 \times 2 \times 1 \\ 5! &= 5 \times 4 \times 3 \times 2 \times 1 \\ 6! &= 6 \times 5 \times 4 \times 3 \times 2 \times 1 \\ 7! &= 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 \\ 8! &= 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 \\ 9! &= 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 \\ 10! &= 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 \end{aligned}$$

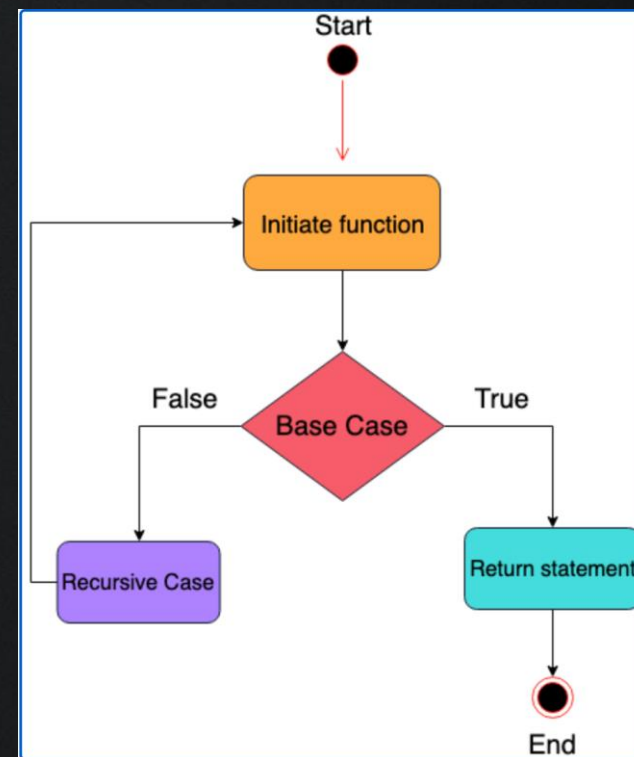
# Estrutura de uma função recursiva

**Caso base:** condição que encerra a recursão, evitando chamadas infinitas.

**Passo recursivo:** chamada da função em um subproblema reduzido, aproximando-se do caso base.

## Fluxo conceitual:

- inicia função
- testa caso base
- se falso, chama-se novamente
- se verdadeiro, retorna valor.



# Exemplo da função recursiva em python

```
def fatorial(n):  
    # Caso base  
    if n == 0:  
        return 1  
  
    # Passo recursivo  
    return n * fatorial(n - 1)  
print(fatorial(5)) # Saída: 120
```

*Caso base: if  $n == 0$ : return 1 — condição de parada.*

*Passo recursivo:  $n * \text{fatorial}(n - 1)$  — reduz  $n$  a cada chamada, garantindo progressão.*

# Fluxo de Execução

- Pilha de chamadas (Call Stack): cada chamada cria um novo quadro (stack frame) na memória.
- Ordem do fluxo: entrada → chamadas recursivas → caso base → retorno acumulado.

Exemplo – Pilha de chamadas para `fatorial(4)`

**Retorno acumulado:**

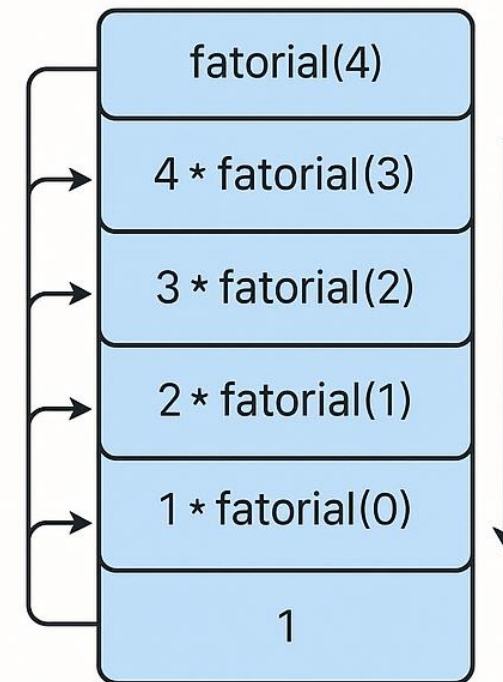
→  $1 \rightarrow 1 * 1 = 1$

→  $2 \rightarrow 1 = 2$

→  $3 \rightarrow 2 = 6$

→  $4 \rightarrow 6 = 24$

Exemplo visual: diagrama da pilha para `fatorial(4)`





# Recursão vs Iteração

## Fatorial

- Iteração: usa laços (for, while) e variáveis acumuladoras.
- Recursão: função chama a si mesma até atingir o caso base.
- Escolha: use a técnica mais eficiente para o problema.

### Iteração

```
def fatorial_it(n):  
    resultado = 1  
    for i in range(1, n+1):  
        resultado *= i  
    return resultado
```

*Geralmente, mais eficiente em memória (evita criar múltiplos stack frames).*

### Recursão

```
def fatorial_rec(n):  
    if n == 0:  
        return 1  
    return n * fatorial_rec(n - 1)
```

*Recursão pode ser mais elegante para problemas naturalmente divisíveis.*

# Como surgem múltiplos stack frames

Quando uma função chama outra, um novo *stack frame* é colocado no topo da pilha.

A pilha vai crescendo até que as funções comecem a retornar e liberar esses frames.

**Recursão** cria *múltiplos frames* porque a função se chama várias vezes **sem ainda terminar as chamadas anteriores**.

Durante a execução de `fatorial(4)`, teremos:

`fatorial(4)` (frame 1)

`fatorial(3)` (frame 2)

`fatorial(2)` (frame 3)

`fatorial(1)` (frame 4)

`fatorial(0)` (frame 5)

# Exercício – Inverter string

1. Crie uma função que receba uma string e retorne a string invertida.
2. Versão recursiva: sem usar métodos prontos como `[::-1]`.
3. Versão iterativa: usando um loop.

# Exercício – Inverter string

## Recursão

```
def inverter_rec(s):  
    # Caso base: string vazia ou 1 caractere  
    if len(s) <= 1:  
        return s  
    # Passo recursivo: inverte o resto + primeiro  
    # caractere  
    return inverter_rec(s[1:]) + s[0]  
  
# Teste  
print(inverter_rec("fiap"))
```

## Iterativa

```
def inverter_it(s):  
    invertida = ""  
    for char in s:  
        invertida = char + invertida  
    return invertida  
  
# Teste  
print(inverter_it("python"))
```

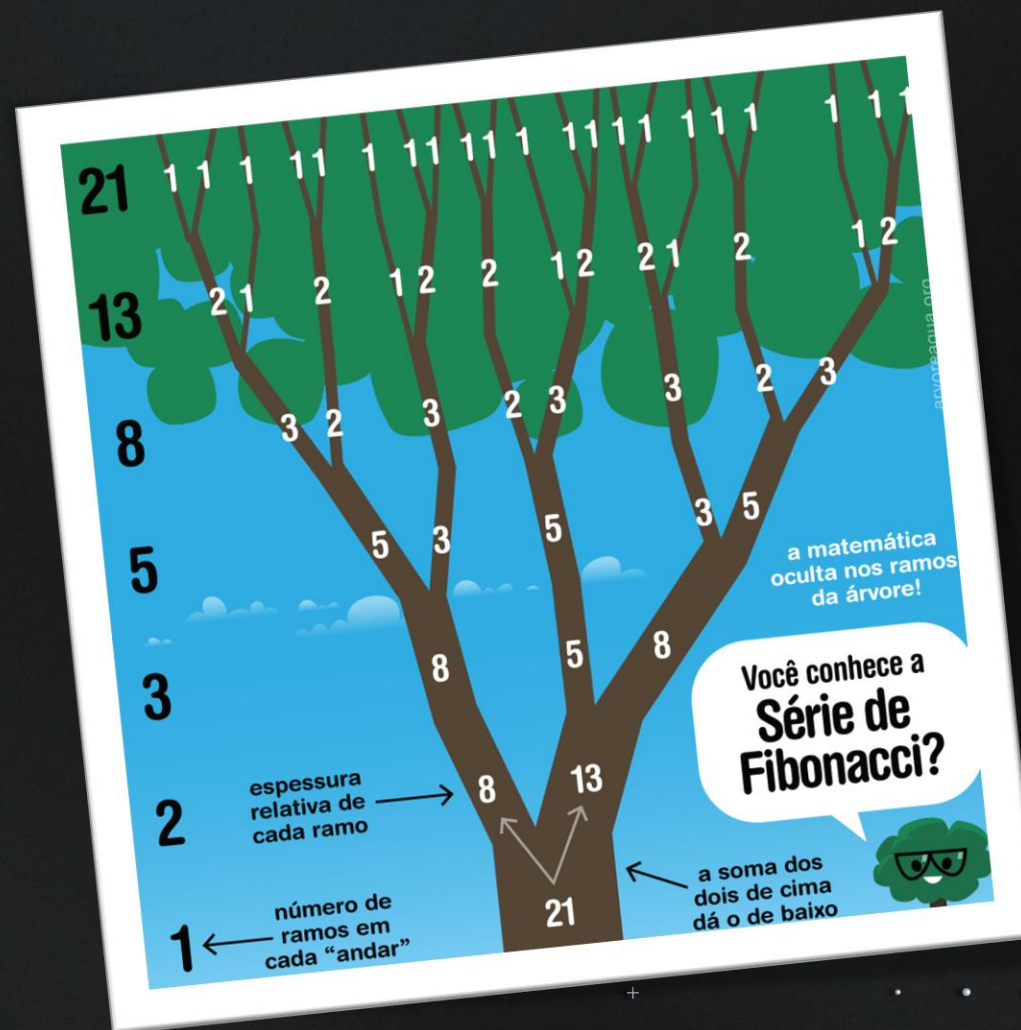


# Definição Matemática: Fibonacci

A sequência de Fibonacci é uma progressão numérica em que cada número é a soma dos dois anteriores.

$F(0) = 0, F(1) = 1$  (casos base)

$F(n) = F(n-1) + F(n-2)$  (passo recursivo)



# Recursão vs Iteração

## Fibonacci

### Iteração

```
def fib_it(n):  
    if n <= 1:  
        return n  
    a, b = 0, 1  
    for _ in range(2, n + 1):  
        a, b = b, a + b  
    return b  
  
# Teste  
print(fib_it(6)) # Saída: 8
```

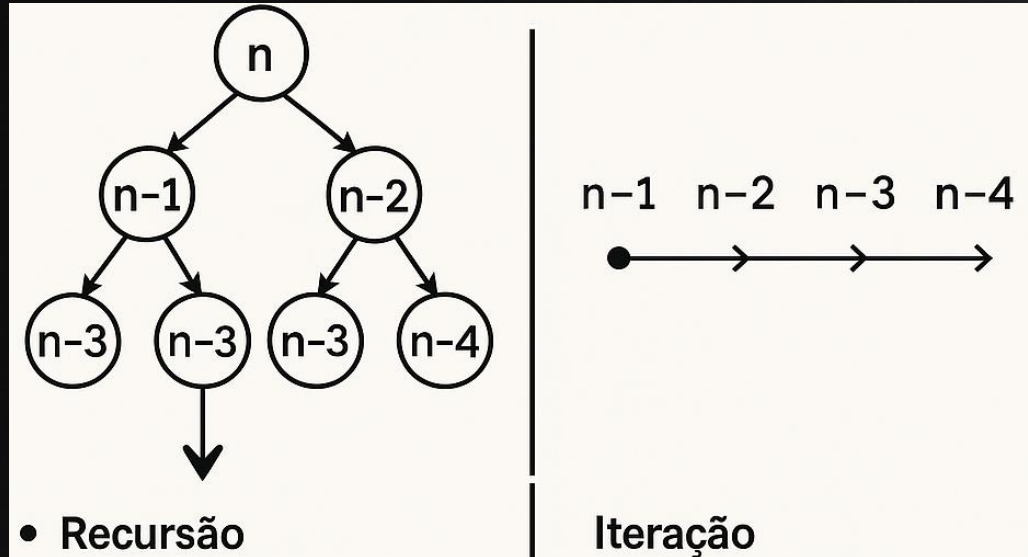
### Recursão

```
def fib_rec(n):  
    # Casos base  
    if n <= 1:  
        return n  
    # Passo recursivo  
    return fib_rec(n - 1) + fib_rec(n - 2)  
  
# Teste  
print(fib_rec(6)) # Saída: 8
```

O `_` está recebendo cada valor do `range(2, n + 1)`, mas não importa o que está ali — o loop só é usado para repetir a operação um número específico de vezes.

# Recursão vs Iteração

## Fibonacci



| Método        | Tempo Estimado | Memória Usada | Complexidade |
|---------------|----------------|---------------|--------------|
| Recursão pura | ~3s            | Alta (stack)  | $O(2^n)$     |
| Iteração      | <0.001s        | Baixa         | $O(n)$       |

*Simples e próxima da definição matemática, mas extremamente ineficiente.*

*Evita recomputação e é muito mais rápida ( $O(n)$ )*

# Recursão vs Iteração

## Conclusão

| Problema  | Recursão pura                  | Iteração         | Melhor escolha |
|-----------|--------------------------------|------------------|----------------|
| Fatorial  | $O(n)$ , mas usa mais memória  | $O(n)$ eficiente | Iteração       |
| Fibonacci | $O(2^n)$ , lento e ineficiente | $O(n)$ , rápido  | Iteração       |

A recursão volta a ser competitiva se for otimizada com memoização ou programação dinâmica, como no Fibonacci com cache, que também vira  $O(n)$  mas mantém a clareza da recursão.



# Exercício Prático 1

## Implementar potencia(base, expoente) recursivamente

Objetivo: criar uma função que calcula  $\text{base}^{\text{expoente}}$  usando recursão.

Caso base: se  $\text{expoente} == 0 \rightarrow$  retorna 1.

Passo recursivo:  $\text{base} * \text{potencia}(\text{base}, \text{expoente} - 1)$ .

```
def potencia(base, expoente):  
    # Caso base  
    if expoente == 0:  
        return 1  
    # Passo recursivo  
    return base * potencia(base, expoente - 1)  
  
# Teste  
print(potencia(2, 5)) # Saída: 32
```

### Vantagens sobre a recursiva:

- Não consome memória extra com stack frames.
- Mais eficiente para expoentes grandes.
- Evita estouro de pilha (stack overflow) em linguagens que limitam recursão.

```
def potencia_iterativa(base, expoente):  
    resultado = 1  
    for _ in range(expoente):  
        resultado *= base  
    return resultado  
  
# Teste  
print(potencia_iterativa(2, 5)) # Saída: 32
```

# Situações em que recursão é preferível

## 1. Estruturas de dados hierárquicas

- Ex.: árvores, grafos, diretórios em um sistema de arquivos.
- Navegar ou processar cada nó de uma árvore é naturalmente recursivo.

```
def soma_arvore(no):  
    if not no:  
        return 0  
    return no.valor + soma_arvore(no.esquerda)  
+ soma_arvore(no.direita)
```

# Situações em que recursão é preferível

## 2. Problemas definidos matematicamente de forma recursiva

- Ex.: Fibonacci (com memoização), fatorial, sequências definidas por relações de recorrência.
- O código recursivo reflete diretamente a definição matemática.

# Situações em que recursão é preferível

## 3. Divisão e conquista (Divide and Conquer)

- Ex.: mergesort, quicksort, pesquisa binária.
- Recursão permite quebrar um problema grande em subproblemas menores até atingir casos simples.



# Situações em que recursão é preferível

## 4. Expressividade e legibilidade

- Recursão pode gerar código mais curto e mais próximo da lógica humana.
- Isso é útil em contextos didáticos ou quando a clareza é mais importante que a micro-otimização.

# Quando evitar recursão

- Quando o problema exige muitas chamadas profundas (risco de *stack overflow*).
- Quando o mesmo problema pode ser resolvido iterativamente de forma mais eficiente.
- Quando a recursão não traz clareza ou simplificação ao código.

# Quando evitar recursão

1. Cada vez que uma função é chamada, o computador guarda:

- variáveis locais, endereço de retorno, estado da execução, em uma estrutura chamada pilha de chamadas (call stack).

2. Essa pilha tem tamanho limitado.

3. Se as funções chamarem outras funções continuamente (como em uma recursão sem fim ou muito profunda), a pilha cresce até ficar cheia.

4. Quando não há mais espaço, ocorre o stack overflow → o programa trava ou encerra com erro.



# Recursion vs Iteration: Explained Simply for Beginners!

## Advantages of Recursion

- Elegant Solution for Complex Problems**  
Some problems are naturally recursive (e.g., tree traversal, graph algorithms), making recursive solutions more intuitive and easier to understand.
- Code Simplicity**  
Recursive solutions are often more concise and easier to write than their iterative counterparts, especially for algorithms with complex control flow.
- Direct Mapping to Mathematical Definitions**  
Many mathematical concepts are recursively defined (e.g., factorial, Fibonacci), making recursive implementations more direct.

### Tree Traversal Example

Recursion makes traversing hierarchical structures like trees simple and intuitive:

```
# Recursive tree traversal (in-order)
def traverse(node):
    if node is None:
        return

    # Process left subtree
    traverse(node.left)

    # Process current node
    print(node.value)

    # Process right subtree
    traverse(node.right)
```

## Recursion vs Iteration: Explained Simply for Beginners!

### Advantages of Iteration

Compartilh...

- Better Performance**  
Iteration is generally faster as it avoids function call overhead and stack operations, leading to better runtime performance.
- Memory Efficiency**  
Uses constant memory regardless of input size, avoiding stack overflow risks with large inputs that recursive solutions might face.
- Easier to Debug**  
Iterative solutions can be easier to trace and debug since the program state is visible in variables rather than hidden in the call stack.

### Fibonacci Example

Iteration dramatically improves performance for Fibonacci calculations:

```
# Iterative Fibonacci
def fibonacci(n):
    if n <= 1:
        return n

    a, b = 0, 1

    for i in range(2, n + 1):
        a, b = b, a + b

    return b
```

MAIS VÍDEOS

<https://www.youtube.com/watch?v=1rWjjO-rZk0&t=314s>



***A tecnologia é melhor  
quando une as pessoas  
(Matt Mullenweg,  
Criador do WordPress)***

# Referências

- John Paul Mueller / Luca Massaron, Algoritmos para leigos, Editora Alta books, 2018 - 1ª edição.
- Additya Y. Bhargava, Entendendo Algoritmos - um guia ilustrado para programadores e outros curiosos, Editora Novatec, 2018, 1ª edição.
- José Augusto N. G. Manzano / Jayr Figueiredo de Oliveira, Algoritmos: Lógica Para Desenvolvimento de Programação de Computadores, 29ª edição, 2019.
- Thomas H. Cormen / Charles E. Leiseson / Ronald L. Rivest / Clifford Stein, Algoritmos - Teoria e Prática, Editora Elsevier, 2012, 3ª edição.